TECNOLÓGICO NACIONAL DE MÉXICO

Instituto Tecnológico de Apizaco

INGENIERÍA INFORMÁTICA

GRUPO: X1

MATERIA:

INTERCONECTIVIDAD DE REDES

DOCENTE:

LUIS ALBERTO LUNA HERNÁNDEZ

SEMESTRE:

5° Semestre

NOMBRE DEL TRABAJO:

Práctica de GitHub - Algoritmo SP

PRESENTA:

Angie Karen Pérez Godínez

No Control: 24371129

FECHA DE ENTREGA:

19/10/2026

Introducción

Tuvimos que documentar un archivo de texto llamado AlgoritmoSP.txt. El documento explicaba la lógica de los protocolos de interconexión, centrándose sobre todo en el árbol de expansión o Spanning Tree Protocol. Para lograr que la copia local y el repositorio en línea quedaran sincronizados, usamos la terminal de comandos. Subimos el archivo siguiendo los pasos adecuados.

Índice

Introducción 2
Reporte de la Práctica 4
AlgoritmoSP.txt ya subido en el GitHub 5
Conclusión 6
Referencias bibliográficas 7

Índice de figuras

Figura 1: Comandos de GitHub 4
Figura 2: AlgoritmoSP.txt en el bloc de notas 4
Figura 3: Carpeta de repositorio 5
Figura 4: AlgoritmoSP.txt en el GitHub 5

Reporte de la Práctica

Una vez comprendimos las funciones básicas del simulador online, el profesor nos pidió realizar una topología similar a la siguiente:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\System32> cd C:\Users\Public
PS C:\Users\Public> cd PracticasDeInterconectividad
PS C:\Users\Public\PracticasDeInterconectividad> git remote add origin https://github.com/akarenpg/Interconectividad-de-
Redes.git
error: remote origin already exists.
PS C:\Users\Public\PracticasDeInterconectividad> git add AlgoritmoSP.txt
PS C:\Users\Public\PracticasDeInterconectividad> git commit -m "Carga del algoritmo SP ya completo"
[main 7f4d8ce] Carga del algoritmo SP ya completo
 1 file changed, 26 insertions(+), 2 deletions(-)
PS C:\Users\Public\PracticasDeInterconectividad> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 18 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 865 bytes | 432.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/akarenpg/Interconectividad-de-Redes.git
   2634b8a..7f4d8ce  main -> main
PS C:\Users\Public\PracticasDeInterconectividad>
```

Figura 1: Comandos de GitHub

AlgoritmoSP.txt — Sin título

Archivo Editar Ver H1

```
Este algoritmo se ejecuta en cada switch para elegir el Root Bridge (puente raíz), determinar los costos
de ruta y desactivar los puertos redundantes.



Algoritmo_STP():
    Para cada switch S en la red:
        Bridge_ID(S) = Prioridad + Dirección_MAC_local(S)
        Root_Bridge = S  // Inicialmente cada switch se cree raíz a sí mismo
        Root_Path_Cost = 0
        Root_Port = NULO


ciclos

    Mientras la red esté activa:
        Enviar BPDU (Bridge Protocol Data Unit) con:
            - Mi_Bridge_ID
            - Root_Bridge_conocido
            - Root_Path_Cost

        Para cada BPDU recibida de un vecino V:
            Si (Root_Bridge_vecino tiene mejor Bridge_ID que mi Root_Bridge actual) O
               (Tienen el mismo Root_Bridge Y Costo_vecino + Costo_enlace < mi_Root_Path_Cost):

                Root_Bridge = Root_Bridge_vecino
                Root_Path_Cost = Costo_vecino + Costo_enlace
                Root_Port = Puerto_hacia_vecino(V)

        // Asignación de roles de puertos
        Para cada puerto P del switch S:
            Si P == Root_Port:
                Estado(P) = FORWARDING  (Puerto Raíz)
            Sino si P es el puerto designado para el segmento (menor costo hacia la raíz):
                Estado(P) = FORWARDING  (Puerto Designado)
            Sino:
                Estado(P) = BLOCKING    (Puerto Bloqueado para evitar bucles)

Ya completo

Prueba
```

Figura 2: AlgoritmoSP.txt en el bloc de notas

AlgoritmoSP.txt ya subido en el GitHub



Figura 3: Carpeta de repositorio

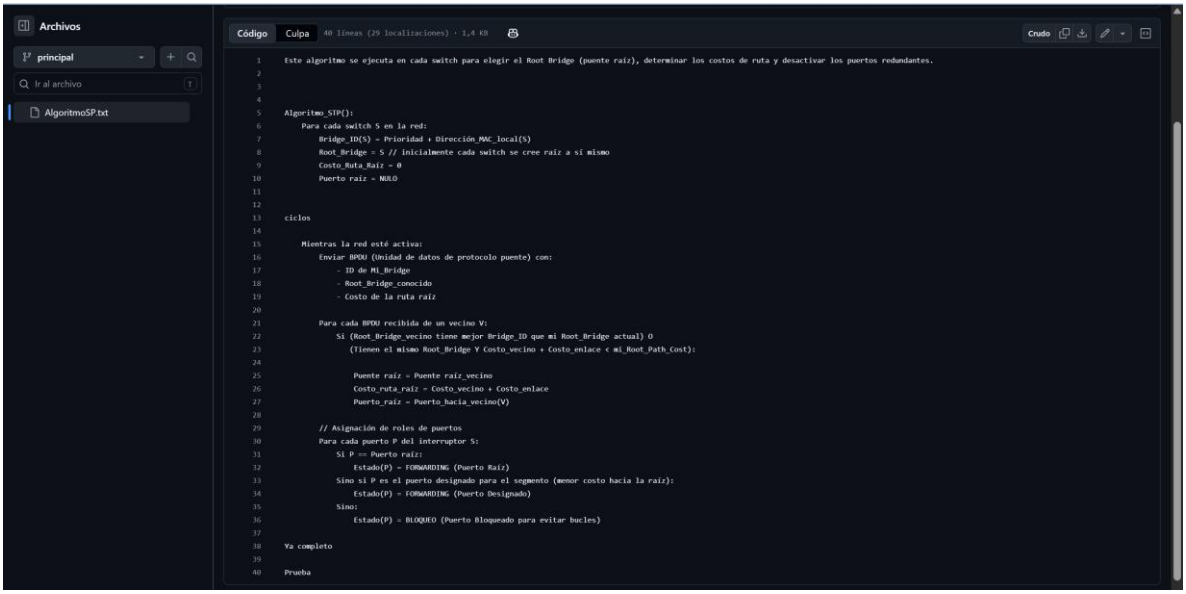


Figura 4: AlgoritmoSP.txt en el GitHub

Conclusión

Poner en práctica estos comandos en la consola, como git add, git commit y git push, sirvió para dominar la gestión de repositorios. Con esta tarea, ahora resulta más sencillo controlar y publicar tanto algoritmos de red como archivos de configuración.

Entender cómo funciona Git ayuda a proteger los datos técnicos y, además, mejora la destreza necesaria para el control de versiones, un aspecto clave tanto en la ingeniería de redes como en el desarrollo de software.

Referencias bibliográficas

- GitHub · Change is constant. GitHub keeps you ahead. · GitHub. (s. f.). GitHub. https://github.com/